# Assignment 1 – Summary

## GitHub repository

| | |
|---|---|
| GitHub username | ishan-monie |
| Repository name | advanced-ml-hw1 |
| Link to the code | https://github.com/ishan-monie/advanced-ml-hw1 |

All of the code used for this assignment is in this repository.

## How to run the code

**Step 1.** Download the repository (green *Code* button, then *Download ZIP*) or clone it, and install the Python packages:

```
pip install numpy matplotlib
```

**Step 2.** Shuttle problem. Open a terminal in the `q1_shuttle` folder and run:

```
python run_q1.py     # solves the problem 3 ways and saves the 3 plots in figures/
python test_q1.py    # runs the 32 edge-case and correctness checks (PASS/FAIL for each)
```

The README in the repository repeats these instructions and explains what each file does. Instructions for the policy gradient questions will be added to the same README.

# Question 2: Value and Policy Iteration (the shuttle problem)

Each period we choose whether to dispatch a shuttle (cost 100, takes up to 15 people) or wait (every person left waiting costs 2). Between 1 and 5 new people arrive each period. We solve this three ways. A lower cost is better in every graph.

## Assumptions

The problem leaves some details open, so these are the choices we made:

- **What we track.** The only thing that matters when deciding is how many people are waiting at the start of a period (0 to 200). We do not track the time of day or what happened before.
- **Costs, not rewards.** Everything is a cost, so we look for the *lowest* total cost. The “value” in the graphs is the expected total cost, so lower is better.
- **Order of events in a period.** We first decide (wait or dispatch), then pay, then the new people arrive.
- **Dispatching.** The shuttle is always available. It takes up to 15 people, or everybody if fewer than 15 are waiting. We allow dispatching with an empty station, even though it is wasteful.
- **The waiting cost.** The cost of 2 per person is charged on the people *left* after the shuttle has gone (everyone, if we wait).
- **Full station.** The station holds at most 200 people. If more would arrive, the extra people are turned away (they are lost, they do not wait). We checked that this choice does not change the answer.
- **Arrivals.** Each period 1, 2, 3, 4 or 5 people arrive, each with chance 1/5, independent of the past.
- **Discounting.** A cost one period later counts 0.95 as much. The cost in the first period is not discounted.
- **The end of the 500 periods (enumeration).** After the last period nothing more is paid, so people still waiting at that point cost nothing extra.
- **The plan.** The best action depends only on the number of people waiting, and it is the same in every period (this is what value iteration and policy iteration find).
- **Stopping and ties.** Value iteration stops when no value changes by more than 1e-10. Policy iteration starts from “never dispatch” and keeps its current action when two actions cost exactly the same.

## (a) Enumeration, 500 periods

We start at the end of the 500 periods, where nothing is left to pay, and work backwards, keeping the cheaper of the two actions at every step. The graph shows the best expected total cost at time 0 for each number of people waiting.

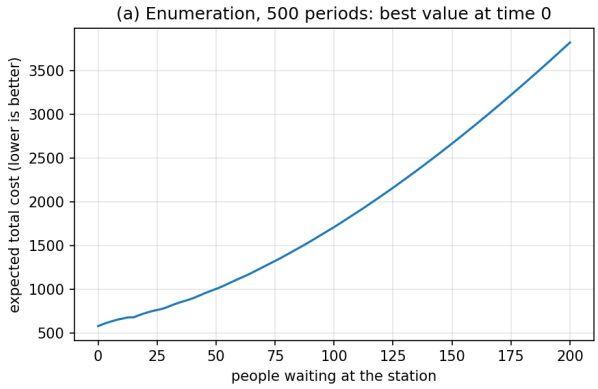



Figure 1. Enumeration: best expected cost at time 0 versus people waiting.

## (b) Value iteration, no end date

The same step is repeated with no end date until the numbers stop changing (519 rounds). The curve is the same as in (a), which is expected: after 500 periods the far future is discounted to almost nothing, and the two differ by about 3 in a billion.

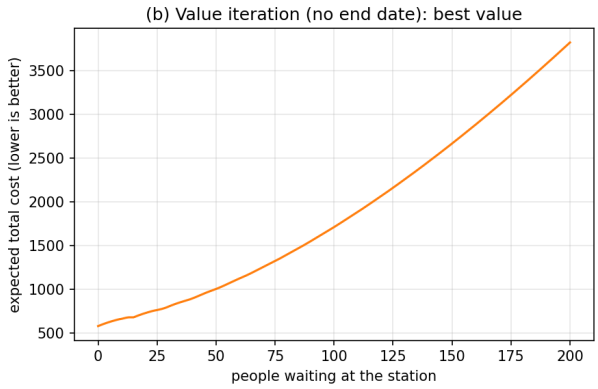



Figure 2. Value iteration: best expected cost versus people waiting.

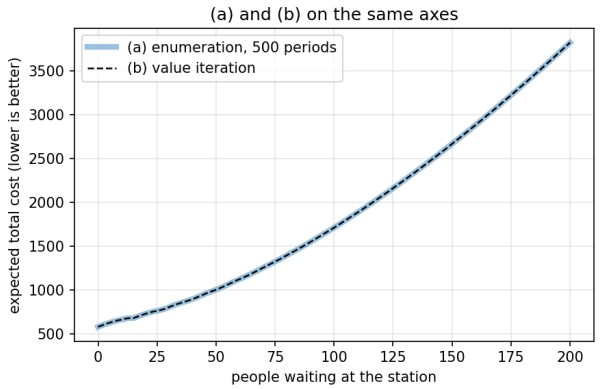



Figure 3. Figures 1 and 2 on the same axes. The two curves lie on top of each other.

## (c) Policy iteration

Here we improve a plan instead of the costs. Starting from “never dispatch”, we measure how good the plan is, change it wherever another action is cheaper, and repeat. It settled after 6 rounds. The best plan is to wait while fewer than 13 people are waiting and to dispatch at 13 or more.

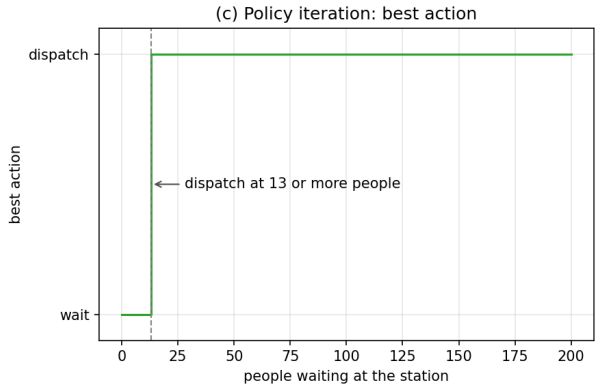



Figure 4. Policy iteration: best action versus people waiting.

# Question 2: Edge cases and tests

An **edge case** is an unusual situation that could break a solution, such as an empty station or a full one. We handled these in two ways, and this section lists both:

• **Part 1 – Protections:** what the code itself does so that unusual situations cannot cause wrong answers (P1 to P17).

• **Part 2 – Tests:** checks that try the unusual situations, and changes to the problem where we can predict the answer, to prove the protections work. The last column says which protection each test covers. All 32 tests pass. Run them with `python test_q1.py`.

## Part 1: Protections in the code

### Built into the calculation

| # | Unusual situation | How the code handles it |
|---|---|---|
| P1 | Shuttle leaves with fewer than 15 people | It takes everybody; the number left is never below 0 (`max(people - 15, 0)`). |
| P2 | Arrivals would push the station past 200 | The station stops at 200 and the extra people are turned away (`min(..., 200)`). |
| P3 | Chances of the next situation not adding up to 100% | Each possible arrival number gets exactly 1 / (number of options), so they always add up to 100%. |
| P4 | Off-by-one: 0 to 200 is 201 situations, not 200 | Every table is sized 200 + 1. |
| P5 | Waiting should not remove anybody | When we wait, everyone stays; we pay 2 for each. |
| P6 | Enumeration needs a starting point | After the last period the value is 0, then it works backwards from period 499 to 0. |
| P7 | Value iteration stopping too early or never | It stops only when no value changes by more than 1e-10, not after a fixed number of rounds. |
| P8 | Policy iteration looping on ties or rounding noise | It only changes an action if the new one is better by more than 1e-9; otherwise it keeps the current action. |
| P9 | Value iteration flip-flopping on exact ties | Dispatch is chosen only if it is strictly cheaper; a tie means wait. |
| P10 | Inaccurate evaluation of a plan | Policy iteration solves the exact equations instead of approximating. |
| P11 | Dispatching an empty station | Allowed, with no special rule. It is just never the best action. |

### Safety checks added to stop bad inputs and runaway calculations

| # | Unusual situation | How the code handles it |
|---|---|---|
| P12 | Nonsense numbers: no arrivals, negative arrivals, sizes or costs, a station limit of 0, tables that do not match the problem | It stops at the start with a clear message saying what is wrong. |
| P13 | A discount of 1 or more (or below 0) | Rejected. With a discount of 1 the “forever” problem has no finite answer. |
| P14 | Value iteration never settles | A maximum number of rounds; if it is exceeded the program stops with an error instead of running forever. |
| P15 | Policy iteration never settles | The same maximum for policy iteration. |
| P16 | A strange horizon for enumeration (0, negative or a fraction of a period) | Rejected: it needs a whole number of periods, at least 1. |
| P17 | A value turns into NaN or infinity | The calculation stops with an error, so bad numbers cannot spread quietly. |

We did not guard against things like typing text instead of a number: that is careless typing, not part of this problem, and the assignment fixes all the numbers.

# Part 2: Tests

## A. Ordinary situations

| Case | What we expected | What we found | Protects |
|---|---|---|---|
| A1. Empty station (0 people) | Wait: a shuttle would cost 100 to carry nobody. | Best action is to wait. | P11 |
| A2. Dispatching with fewer than 15 people (4) | The shuttle takes everybody; we pay only the 100. | Cost 100; next period has 1 to 5 people. | P1 |
| A3. Does the 200-person limit matter? Try 300. | No change to the best actions. | Same actions up to 150 people; costs differ by 9e-11. | P2 |
| A4. Chances of the next situation add up to 100% | Exactly 100% in every situation. | Largest error 0. A deliberately damaged table is caught by the same check. | P3 |
| A5. A bigger shuttle | Wait for more people before sending it. | Shuttle of 5 / 15 / 30 dispatches at 5 / 13 / 17 people. | – |

## B. Edges of the station

| Case | What we expected | What we found | Protects |
|---|---|---|---|
| B1. Exactly 15 people, then 16 | 15 empties the station; 16 leaves one behind. | Costs 100 and 102. | P1 |
| B2. Full station (200) | Dispatch, and waiting cannot push it above 200. | Best action is to dispatch; waiting stays at 200. | P2 |
| B3. Nearly full (199 and 198) | At 199 every arrival is turned away. At 198 only one fits. | From 199: always 200. From 198: 20% to 199, 80% to 200. | P2 |
| B4. Off-by-one | 201 situations (0 to 200), not 200. | All tables are 201 wide and the graphs have 201 points. | P4 |

## C. Arrivals

| Case | What we expected | What we found | Protects |
|---|---|---|---|
| C1. Nobody ever arrives, 10 waiting (worked by hand) | Waiting forever costs 20/(1-0.95) = 400; dispatching once costs 100. Cheapest: 100. | The program gives 100.000. | – |
| C2. Far more arrivals than the shuttle can carry (16 to 20 per period) | The queue never empties, so each person left costs 2/(1-0.95) = 40 forever. A dispatch (100) pays off from 100/40, so from 3 people. (Our first guess here was 15; working it through showed the guess was wrong, not the program.) | Dispatches for every number from 3 to 152. | – |
| C2b. Why does it stop at 152? | Near the limit the queue hits 200 anyway, so a dispatch saves little. The point should move if the limit moves. | Last dispatch at 152 (limit 200) and 252 (limit 300): an effect of the cap. | P2 |

## D. Extreme prices

| Case | What we expected | What we found | Protects |
|---|---|---|---|
| D1a. Free shuttle | Dispatch whenever anyone is waiting. | Dispatches in 200 of 200 situations. | – |
| D1b. Free waiting | Never dispatch; total cost 0. | Never dispatches; cost 0. | – |
| D2. Enormous shuttle cost (1,000,000) | Never dispatch, even with a full station. | Never dispatches. | – |

## E. Time and discounting

| Case | What we expected | What we found | Protects |
|---|---|---|---|
| E1. Very short horizon (worked by hand) | 1 period left costs 2 x people. 2 periods left costs 3.9 x people + 5.7. | Both match the program exactly. | P6 |
| E2. Is 500 periods long enough? Compare 100, 200 and 500. | The answer settles down as the horizon grows. | Difference from the no-end-date answer: 4.0 / 0.024 / 0.000000003. | P6 |

## F. Checks on each method

| Case | What we expected | What we found | Protects |
|---|---|---|---|
| F1. Enumeration: the starting point | Nothing left to pay after the last period; more periods left never costs less. | Value after period 500 is 0 everywhere; costs only grow with more periods. | P6 |
| F2. Value iteration: different starting guesses | Same answer from any guess. | All 0, all 1000 and random guesses agree to 4e-9. | P7 |
| F3. Policy iteration: improving and ties | Each round improves the plan. When all actions cost the same, it must not loop forever. | 6 rounds, costs never went up. The all-ties problem stopped after 1 round. | P8, P9 |

## G. Overall checks

| Case | What we expected | What we found | Protects |
|---|---|---|---|
| G1. All three methods agree | Same costs and same plan. | Biggest gap 3e-9; identical plan. | all |
| G2. Policy iteration from a bad start (“always dispatch”) | Still finds the same best plan. | It does. | P8 |
| G3. Simulate the station 3000 times | The average cost matches the calculated cost. | Simulated 575.8 (±1.7) against calculated 576.7. | – |

## H. The safety checks themselves

| Case | What we expected | What we found | Protects |
|---|---|---|---|
| H1. Nonsense numbers (no arrivals, negative arrivals, sizes or costs, limit 0) | Each one is rejected with a clear message. | 6 of 6 rejected. | P12 |
| H1b. Tables of the wrong size, or whose chances do not add to 100% | Rejected. | Both stopped. | P12 |
| H2. Discount of 1.0, 1.5 or -0.1; and 0.0 | The first three are rejected; 0.0 is allowed. | As expected. | P13 |
| H3. Value iteration with a target it can never meet | Stops with an error after the maximum rounds. | It does (50 rounds allowed). | P14 |
| H4. Policy iteration allowed only 1 round (it needs 6) | Stops with an error. | It does. | P15 |
| H5. Enumeration with 0, -3 or 2.5 periods | Rejected. | All three rejected. | P16 |
| H6. A NaN or infinite cost | All three methods stop with an error. | They do. (An infinite cost on an action nobody uses is, rightly, not an error.) | P17 |
| H7. Waiting with 10 people | We pay 20; next period has 11 to 15 people. | Cost 20; chances 20% each for 11 to 15. | P5 |
| H8. Exact equation solving vs a slow repeat-until-settled calculation | Same costs. | Largest gap 9e-13. | P10 |

# Question 2: With 3 more months, what else would we test?

Everything so far checks that the program solves *our model* correctly. With more time, the bigger question is whether the model is a good picture of a real shuttle station, and whether the answer can be trusted if things are a bit different.

- **Use real data for arrivals.** We assumed 1 to 5 people arrive with equal chance. With real counts we would fit the true pattern and see whether the “dispatch at 13” rule still holds.
- **Rush hour and quiet times.** Arrivals change during the day. We would add the time of day to the model and see how the best rule changes.
- **What if our prices are a bit wrong?** The costs 100 and 2 are estimates. We would try, say, 50% to 150% of each and plot how the dispatch point moves, to see how sensitive the answer is.
- **Compare with simple rivals.** Simulate “dispatch at 10”, “at 15”, “at 20” and “every period” for a long time and show with confidence ranges that 13 really is the cheapest.
- **Test on hundreds of random problems.** Generate random shuttle sizes, prices and arrival patterns automatically, and check on each that all three methods agree and that a brute-force search over every plan on small versions gives the same best plan.
- **A more realistic shuttle.** Travel time (the shuttle is away for a while), several shuttles, people who give up waiting, groups arriving together, and a larger station. We would also check the program stays fast and accurate as the station gets much bigger.
- **Find the answer without being told the rules.** Let a learning method (like the policy gradient methods in the next questions) learn from simulated stations only, and check that it also finds about 13. That would be an independent check on our model.
- **Ask someone who runs a station.** Check that our assumptions (for example, that the waiting cost is charged after the shuttle leaves) match how it really works.